

Advanced Database Management Systems (Advanced DBMS)

Capstone Project — MiniDB Final System Guidelines

Overview

The Capstone Project is the most important component of the Advanced DBMS course.

Throughout the course, students build individual database components including storage engines, indexing structures, query processing, transaction management, concurrency control, and recovery mechanisms. The capstone project brings all of these components together into a coherent, working relational database system.

The goal is to design, implement, and demonstrate a functioning MiniDB system that incorporates the concepts studied across all lab modules.

Unlike traditional assignments, this project evaluates system integration, architectural understanding, engineering trade-offs, and the ability to explain design decisions.

Students will work in teams and submit their project through GitHub along with documentation and a project demonstration.

Project Objective

The objective is NOT to:

- Build a feature-rich database with hundreds of commands.
- Copy existing open-source database implementations.
- Generate code without understanding the underlying concepts.
- Optimize for lines of code rather than correctness and design.

You are expected to:

- Build a working database engine from foundational components.
- Integrate storage, indexing, query processing, transactions, and recovery.
- Demonstrate understanding of database internals.
- Make sound engineering decisions and explain trade-offs.
- Deliver a functioning system that can be evaluated through demonstrations and discussion.

Marks will primarily be awarded based on:

- Correctness
 - Completeness
 - System design
 - Engineering quality
 - Understanding demonstrated during the viva
-

Team Formation

Team Size

Teams must consist of:

- Minimum: 2 students
- Maximum: 4 students

Students are free to choose their teammates.

Project Deliverables

Each team must deliver:

1. Working MiniDB System

A functioning database engine implementing the required features.

2. Source Code

Well-structured and documented codebase.

3. Project Documentation

README.md containing:

- Architecture overview
- Design decisions
- Implementation details
- Benchmark results
- Setup instructions

4. Benchmark Report

Performance measurements and analysis.

5. Project Demonstration

Live demonstration of system functionality.

6. Viva Discussion

Each team member should be able to explain:

- Their implementation
 - Design decisions
 - Trade-offs
 - Internal architecture
-

Required Core Features

Every project must implement the following components.

Storage Engine

Implement:

- Page-based heap files
- Page manager
- Buffer pool

Students should demonstrate:

- Page allocation
 - Page reads and writes
 - Buffer pool usage
-

B+ Tree Indexing

Implement:

- Primary key index
- one secondary index optional

Operations:

- Search
- Insert
- Delete

Students should demonstrate index utilization during query execution.

SQL Query Execution

Support at minimum:

SELECT

Including:

- WHERE
- JOIN
- INSERT

DELETE

End-to-end query execution should be demonstrated.

Cost-Based Optimizer

Implement at least:

- Selectivity estimation
- Join order selection

The optimizer should be able to choose between:

- Table scan
- Index scan

where applicable.

Transaction Management

Implement:

- Serializable isolation
- Two-Phase Locking

Students should demonstrate:

- Concurrent transactions
 - Lock acquisition
 - Deadlock scenarios
-

Recovery

Implement:

- Write Ahead Logging (WAL)
- Crash recovery

Students must demonstrate:

- System crash
 - Recovery process
 - Preservation of committed transactions
-

Extension Tracks

Each team must choose ONE extension track.

Track A — Performance

Objective

Improve execution efficiency.

Possible approaches:

- Vectorized execution engine
- Columnar storage layer
- Batch processing optimizations

Benchmark Requirement

Compare against:

- Original row-store implementation

Analyze:

- Query latency
 - Throughput
 - Resource utilization
-

Track B — Concurrency

Objective

Replace 2PL with MVCC.

Implement:

- Multi-Version Concurrency Control
- Snapshot visibility rules
- Version management

Demonstration Requirement

Show:

- Higher read throughput
 - Reduced blocking under contention
 - Concurrent transaction behavior
-

Track C — Modern Storage

Objective

Replace heap-file storage with an LSM-tree based design.

Possible components:

- MemTable
- SSTables
- Compaction

Benchmark Requirement

Compare against:

- B+ Tree based storage

Analyze:

- Write throughput
 - Read latency
 - Storage amplification
-

Track D — Distributed Systems

Objective

Implement a simple replication layer.

Implement:

- Two-node architecture
- Primary-replica model

Demonstrate:

- Replication flow
 - Read consistency
 - Failure scenarios
-

Suggested Milestones

The following milestones are recommended to help teams track progress.

Milestone	Deliverable
M1	Page manager + buffer pool integrated
M2	B+ tree + parser connected
M3	Query execution engine with joins and aggregation
M4	Transactions and locking
M5	Recovery, benchmarking, and final demo

These milestones are intended as guidance and will not be separately graded.

Repository Submission

All projects must be submitted through GitHub.

Students should raise a Pull Request against the designated repository.

The GitHub Pull Request itself will be treated as the official submission.

<https://github.com/KnightKnight27/scaler-Adv-DBMS>

Submission Deadline

Final Submission Deadline

23 June 2026 (11:59 PM IST)

Important:

- No late submissions will be accepted.
 - Teams are strongly encouraged to submit before the deadline.
 - Last-minute submissions may face review delays.
-

Pull Request Guidelines

Repository Structure

Create a project directory using your team name.

Example:

MiniDB_Projects/

```
|— Team_StorageMasters/  
|  
|— README.md  
|— src/  
|— benchmarks/  
|— docs/
```

or

MiniDB_Projects/

```
|— Team_BufferPool/  
| |— README.md  
| |— src/  
| |— benchmarks/  
| |— docs/
```

Pull Request Title Format

The Pull Request title must follow:

TEAM_<TEAM_NAME>

Examples:

TEAM_BufferPoolBuilders

TEAM_QueryOptimizers

TEAM_StorageMasters

Team Information

Along with the Pull Request, teams must clearly mention:

Team Name

Team Members

For every member provide:

- Full Name
- Scaler Email ID
- Roll Number

Example:

John Doe

Roll Number: SCALER12345

Email: john@scaler.com

This information should be included in the README.md and/or PR description.

Submit your PRs to this repo

<https://github.com/KnightKnight27/scaler-Adv-DBMS>

README Requirements

The README.md must include:

1. Project Overview

- Problem statement
 - Goals
 - Chosen extension track
-

2. System Architecture

Include:

- Architecture diagram
 - Major modules
 - Data flow
-

3. Storage Layer

Explain:

- Page format
 - Heap files
 - Buffer pool
-

4. Indexing

Explain:

- B+ Tree design
 - Node structure
 - Search path
-

5. Query Execution

Explain:

- Parser
 - Query plan generation
 - Operator execution
-

6. Optimizer

Explain:

- Cost estimation
 - Selectivity estimation
 - Join ordering
-

7. Transactions & Concurrency

Explain:

- Locking strategy
 - Isolation guarantees
 - Deadlock handling
-

8. Recovery

Explain:

- WAL design
 - Log records
 - Crash recovery procedure
-

9. Extension Track

Explain:

- Motivation
 - Design
 - Results
-

10. Benchmarks

Include:

- Experimental setup
 - Results
 - Analysis
-

11. Limitations

Discuss:

- Missing features
 - Scalability limits
 - Future improvements
-

12. How to Run

Include:

- Build steps
- Dependencies
- Example commands

Viva Process

After submission, projects will undergo a viva evaluation.

The viva may be conducted:

- Offline
- Online

depending on logistics and scheduling.

Purpose of the Viva

The viva is intended as a sanity check to ensure that:

- Students understand the code they submit.
- Students understand the architecture they implemented.
- Students can explain key database concepts used in their system.
- The project is functional and genuinely developed by the team.

This is not intended to be a difficult theoretical examination.

However, students should be prepared to explain:

- Their implementation choices
 - Major modules
 - Trade-offs
 - Benchmark results
 - Demonstrated functionality
-

Evaluation Criteria

Component

Weightage

Evaluation Criteria

Core Feature Completeness	40%	Correctness and completeness of required database features
Extension Track Implementation	20%	Quality and depth of chosen extension
Performance Benchmarks	15%	Quality of experiments and analysis
Code Quality & Architecture	15%	Modularity, readability, maintainability
Final Demo & Report	10%	Communication, explanation, design rationale

Total: 100 Marks

Important Rules

- Teams must consist of 2–4 students.
 - Every project must implement all required core features.
 - Exactly one extension track must be selected.
 - GitHub Pull Request is the official submission.
 - Team member details must be clearly provided.
 - Students should be prepared for a viva discussion.
 - All submitted work should be understood by the team members presenting it.
 - Plagiarism may result in penalties.
 - AI-assisted development is acceptable only if students fully understand, explain, and defend the code they submit.
-

Final Reminder

Building a database system is one of the most challenging and rewarding systems projects in computer science.

The objective of this capstone is not merely to produce code, but to demonstrate an understanding of how real databases are engineered.

Focus on:

- Correctness

- Simplicity
- Architectural clarity
- Trade-off analysis
- Engineering rigor

A well-designed and thoroughly understood MiniDB is significantly more valuable than a larger system whose implementation cannot be explained or defended during the project discussion.